LEXON WORKSPACE and LEXON TEAM

Complete system guide: architecture, backend, frontend, security and operations

Private company operating system (Workspace) and connected member app (Team), built on one Supabase backend.

Package contents: lexon-workspace.zip, lexon-team.zip, LEXON_ALL_IN_ONE.sql, this guide.

1. Overview and honest status

Lexon Workspace is the owner/admin control center. Lexon Team is a separate app for members. Both are static web apps (also installable as PWAs) that talk to the same Supabase project. There is no custom application server: security and business rules live in the database (Row Level Security, triggers, functions) and in a few Edge Functions.

What is built

Area	Status
Login (Google for Workspace, email code for Team), TOTP 2FA, sessions, login history, idle logout	Built
Home, Team (invites with status tracking, profiles), Projects (Active / Inactive / Coming Soon), Tasks (All and My Tasks), Windows-style Files explorer, Analytics with multi-chart dashboards	Built
Messages: DMs, groups, files, reactions, reply/edit/delete, typing, unread, realtime	Built
Voice and video calls (WebRTC, Realtime signaling), call history	Built (needs real-device testing)
Notifications and activity events across Workspace and Team	Built
Permissions matrix (Settings), field-level protection of identity data	Built
Integrations registry (OAuth connect/disconnect, status, logs, Vault tokens)	Built (needs your OAuth credentials)
YouTube Automation: sync, analytics, comparison, AI Studio, web research (own Supabase project)	Built (needs Google + Anthropic credentials)
Four-project backend: Workspace+Team shared, Office, Instagram, YouTube separate; secure project-gateway between them	Built (SQL for all four)
Security hardening: anon revoked, rate limits, restrictive 2FA gate, CSP, lazy protected routes, Vault-held project keys	Built
Assistant on Home (Google Gemini, workspace-aware, voice input/output where the browser supports it)	Built (needs a Gemini key)
Lexon Team app (Home, Messages, Activity, Settings, calls, PWA)	Built
Agents module	Architecture only, intentionally empty
Office	Separate environment: its own backend link and AI/API integrations; the Office project itself is not provided yet
Instagram Automation	Card + tables + plan only; waiting for your Lexon Automation project
Desktop shells (Electron) for Windows and macOS, same web build, same backend	Built (packaging must be run on Windows / macOS)

Note. This code was written in a sandbox without internet access. It has NOT been compiled with npm, the SQL has NOT been executed on a Supabase instance, and the Edge Functions have NOT been called against live Google/Anthropic APIs. Expect to fix a few build, SQL or API errors on first run. Section 13 lists the verification steps and known limitations.

2. Architecture

Clients	Supabase A: Workspace + Team (shared)	External services
Lexon Workspace: web/PWA + Windows/macOS desktop Lexon Team: web/PWA (desktop-ready)	Auth (Google, email code, TOTP) Postgres + Row Level Security Storage (files, avatars) Realtime (changes, broadcast) Edge Functions (Deno) Vault (encrypted tokens)	Google OAuth, GitHub OAuth STUN/TURN (calls)

Project	Contains	Reached by
A: Workspace + Team	Everything shared: auth, team, tasks, messages, files, notifications, activity, calls, permissions	Both apps directly (anon key + RLS)
B: Lexon Office	Virtual office members, agents, layout, events (not linked to Workspace people)	Office app (its own auth); Workspace via project-gateway (status and events only)
C: Instagram Automation	accounts, rules, events, actions, threads/messages, analytics, logs, Vault tokens	Lexon Automation app (its own auth); Workspace via project-gateway
D: YouTube Automation	connection, channel snapshots, videos, AI outputs, Vault tokens, Edge Functions (OAuth, sync, AI)	Only Workspace via project-gateway (no client policies at all)

Design principles

- **Database is the authority.** The browser only holds the public anon key. Every read/write is checked by RLS, so a modified frontend gains nothing.
- **No custom server to secure.** Edge Functions exist only where secrets are needed (OAuth, YouTube, AI, invitations). They re-verify the caller and permission themselves.
- **Two apps, one backend.** Workspace and Team share code (auth, messaging, calls, services) but ship separately so members never receive owner/admin screens.
- **Everything real.** Home, analytics, assistant, presence, notifications and YouTube numbers come from stored data. Empty states are shown instead of fake data.

Repository layout

```
lexon-workspace/
  apps/web/            React + TypeScript + Vite + Tailwind
  supabase/migrations/ 0001 ... 0006 SQL (version controlled)
  supabase/functions/  integrations-oauth, youtube-sync,
                       youtube-ai, invite-member, _shared
  docs/                MANUAL_SETUP.md, INSTAGRAM, OFFICE
lexon-team/
  apps/team/           member app (same stack)
```

3. Backend in detail

3.1 Database schema (PostgreSQL)

Group	Tables
Identity and access	profiles, profile_private, workspaces, workspace_members, authorized_emails, role_permissions
Work	projects, project_members, tasks, task_assignees, attendance

Group	Tables
Files	folders, files (+ Storage buckets workspace-files, avatars)
Messaging and calls	conversations, conversation_members, messages, call_sessions
Events and audit	notifications, activity_logs, security_events
Integrations and automation	integrations, integration_secrets, integration_logs, automations, automation_logs
Agents	agents, agent_events (empty by design)
Other projects	Office, Instagram and YouTube data are NOT here: each has its own Supabase project and schema (section 7)

Design notes: private personal data (phone, email, address, emergency contact, notes) lives in **profile_private**, separate from the profile that appears on cards. Roles are stored per workspace in workspace_members; what a role may do is stored in role_permissions, so permissions are editable without code changes.

3.2 Row Level Security (RLS)

Every table has RLS enabled. Typical rules: workspace members can read workspace data; writes require a permission checked by `has_permission(workspace, permission)`; conversations and messages are visible only to conversation members (even the owner cannot read other people's DMs); call rows are visible only to caller and callee; `integration_secrets` has RLS enabled with no policies, so no client can ever read it.

3.3 Functions, triggers and RPCs

Name	Purpose
has_permission, is_member, has_role, in_conversation	Security-definer helpers used by policies (avoid RLS recursion)
mfa_ok	True if session is aal2 or the user has no verified 2FA factor; used by restrictive policies
handle_new_user (trigger on auth.users)	Allow-list gate: first user becomes Owner; later users must be in authorized_emails; creates profile, private row and membership
guard_profile_update, guard_private_update, guard_call_update	Field-level rules: members cannot change name, role title, department, rating, phone or email; call participants are immutable
task_normalize, task_project_sync	Done means 100 percent; project progress is the average of its tasks
task_activity, project_activity, automation_log_activity	Write Recent Activity events automatically
notify_task_assignee, notify_new_message, notify_call	Create notifications for assignees, message recipients, callees
my_conversations, start_direct, create_group, toggle_reaction	Messaging RPCs (list with unread, create chats safely, reactions)
set/get/delete_integration_secret	Vault wrappers; executable only by service_role (Edge Functions)
is_admin, can_see_task	Admin check; Team Members only see tasks assigned to them
list_invites, list_admins, remove_admin, broadcast_notification	Invite lifecycle status, approved-Admin list, owner-only removal, announcements with audience
run_retention_cleanup, set_security_retention	Deletes attendance/activity/automation history older than 7 days and login history per setting (7 to 30 days); scheduled with pg_cron when available

3.4 Roles and access

Workspace has exactly two roles: **Admin** and **Team Member**. The first account (owner) is an Admin; further Admins must be approved explicitly by the owner (Settings, Admin access). There is no permission editor and no Manager role.

Area	Admin	Team Member
Team, Analytics, Automations, Integrations, Agents, Office, Settings (access, workspace)	Full access	Not available
Projects	Create, edit, delete, change status	View
Tasks	All tasks, create, edit, delete	Own assigned tasks only (My Tasks); can update their progress
Files	Everything including delete	Browse, upload, download, rename, move
Messages, calls, notifications, assistant	Yes	Yes (assistant sees only data they may see)
Security (2FA, sessions)	Yes, plus login-history retention	Yes

Admins pass every permission check automatically in the database (has_permission). Team Members pass none of the admin checks. The same rules apply to the Lexon Team app because both use project A.

3.5 Storage

- **workspace-files** (private): path starts with the workspace id; read/write require membership, delete requires files.delete. Access to files is through short-lived signed URLs.
- **avatars** (public read): profile photos only; a user may write only inside their own folder (or with team.edit).
- Chat attachments are stored in workspace-files and tagged so the Files page does not list them.

3.6 Realtime

Realtime is used for: new messages and typing indicators, notifications (bell badge), call invitations and status, presence/tasks for the Office feed. Database changes are delivered only if the subscriber passes RLS. Typing and call signaling (SDP/ICE) use Realtime broadcast channels.

3.7 Edge Functions (Deno)

Function	What it does	Auth
integrations-oauth	Google/Drive/Calendar/Gmail/GitHub: signed expiring state, code exchange, tokens into Vault, disconnect with revoke	JWT checked in code + integrations.manage; callback verified by HMAC state; rate limited
project-gateway	link/unlink a separate project (key verified then stored in Vault); select, upsert, invoke on whitelisted tables/functions; ping (schema check); pull_events (Instagram/Office events into the unified feed)	JWT + integrations.manage (link) or automations.manage (data); 120 requests/min per user
invite-member	invite / resend / revoke: allow-lists the email as Team Member, sends the invitation, and always returns a one-time code and link for the admin to share	JWT + team.invite; 30 per hour
ai-keys	Saves AI/API keys (Gemini, 27-provider catalog, custom APIs) into Vault; test and remove; blocks private/internal endpoints	JWT + integrations.manage; rate limited

Function	What it does	Auth
assistant	Google Gemini chat. Reads workspace context with the caller own session (RLS-limited) and logs assistant activity	JWT + membership; 30 requests/min
(Project D) youtube-oauth, youtube-sync, youtube-ai	Run inside the YouTube project: Google OAuth, sync via YouTube Data/Analytics APIs, AI Studio via Anthropic API	Only callable with that project service key (constant-time compare); OAuth callback needs signed state

3.8 Key data flows

Assigning a task

Owner creates a task and picks assignees. Rows go into tasks and task_assignees. A trigger writes a notification for each assignee and an activity log row. Realtime pushes the notification to the member's Team app bell and Home. When the member sets progress to 100 percent, task_normalize marks it Done, task_project_sync updates the project progress, task_activity logs completion, and Workspace Home and Analytics reflect it.

Sending a message

Message inserted (RLS: sender is member of conversation). Trigger notifies other members. Both chat panes receive a Realtime change event and reload; the unread badge uses last_read_at per member.

Making a call

1) Caller gets microphone/camera and inserts a call_sessions row (status ringing). 2) A trigger notifies the callee and Realtime delivers the row to the callee, whose app shows Accept/Decline. 3) On accept, callee joins the call channel, sets status accepted and sends "ready". 4) Caller creates the WebRTC offer, callee answers, ICE candidates are exchanged over the broadcast channel. 5) Media flows directly peer-to-peer (or via TURN). 6) Either side ends: status becomes ended, both clean up. Unanswered calls become missed after 45 seconds. History shows direction, type, status and duration.

YouTube sync and AI

Workspace calls project-gateway with its normal session. The gateway verifies the JWT and automations.manage in project A, loads the YouTube project key from Vault, and invokes youtube-oauth / youtube-sync / youtube-ai inside project D. Results land in project D tables; Workspace reads them back through the gateway. AI Studio sends only a compact summary of stored numbers plus your input to the model, instructed to cite data and admit gaps. Upload timing is computed from your own publish times and views (UTC).

4. Frontend in detail

4.1 Stack and structure

React 18, TypeScript, Vite, Tailwind CSS, React Router (hash routing for static hosting), supabase-js. Charts are dependency-free SVG components. Structure per app: `auth/` (provider, route guard), `lib/` (env, supabase client, permissions, presence, calls), `services/` (all database/API access), `pages/` (screens), `components/` (shared UI), `layouts/`.

- **Branding** is centralized in lib/brand.ts.
- **Environment validation:** missing Supabase variables show a clear developer message instead of a blank page.
- **Every page** has loading, empty and error (with retry) states. Layouts are responsive; Team has a bottom navigation bar on phones.
- **Permissions in the UI** (useWorkspace().can) only hide controls; the database enforces them.

4.2 Workspace screens

Screen	Highlights
Home	KPI cards, completion area chart, upcoming tasks, active projects, team presence, activity, quick actions, assistant card that opens a Gemini chat panel (renamable, voice)
Team	Member cards, search, profile with tabs, Invite with a status dashboard (Invited, Joined, Pending, Not registered), resend/new code, revoke
Projects / Tasks	Projects: name, type, URL or domain, optional repository, description (up to 5000 words), status Active / Inactive / Coming Soon, combined Links section. Tasks: grouped list, multi-assignee, All and My Tasks
Messages	Conversation list (pinned, unread), chat with files, reactions, reply/edit/delete, search, typing, call buttons, call history
Files	Folders, upload, preview, download, rename, move, delete, search, sort, storage used
Analytics	7/30/90-day completion, created tasks, activity, status split, project progress, most active members
Automations	Instagram card (awaiting import) and YouTube Automation (Overview, Videos with comparison, AI Studio)
Integrations	Cards per provider: connect/disconnect, status, account, last sync, permissions, errors, logs
Agents / Office	Agents: honest empty state. Office: separate environment with its own backend link and AI integrations
Notifications	Bell in the header is the single notification center: messages, files, automations, team, tasks, agents, assistant, with filters and an admin New button for announcements (audience options)
Settings	Security (2FA, sessions, login history and retention), Admin access (owner-approved Gmail allow-list), Workspace (name, retention info)

4.3 Lexon Team screens

- **Login:** email, six-digit code, then 2FA if enabled. Only authorized emails can ever create an account.
- **Home ("Your Work"):** pending tasks, assigned projects, overall progress, notifications, recent activity; members can update progress/status of tasks assigned to them.
- **Messages:** same messenger as Workspace, including files, notifications, realtime and calls (Workspace and Team talk to each other).
- **Activity:** pending work plus a chronological card timeline of assigned, created and completed work with timestamps.
- **Settings:** editable: profile photo, about, address, theme (light/dark/system), 2FA, sessions. Read-only: name, role, department, email, phone (owner-controlled and protected in the database).

4.4 PWA and desktop readiness

Both apps ship a web manifest and a minimal service worker that caches only static build assets, never API or user data. They are plain static builds, so a future Electron shell can load the same code with a preload/IPC layer; no business logic is duplicated.

5. Security model

5.1 Defense in depth

Layer	Control
Account creation	Allow-list trigger: unauthorized emails cannot create an account at all. First user is Owner.

Layer	Control
Authentication	Google (Workspace) or email one-time code (Team) via Supabase Auth; short-lived JWT plus refresh token.
Second factor	TOTP 2FA. The login screen demands the code, and restrictive RLS policies (mfa_gate) make the database refuse data to non-aal2 tokens for users who enabled 2FA, even through direct API calls.
Authorization	Role + permission matrix checked inside RLS and Storage policies. Frontend checks are cosmetic.
Field-level protection	Triggers stop members changing name, role title, department, rating, phone and email. Personal data sits in a separate restricted table.
Sessions	Logout (this device), sign out other devices, sign out everywhere (revokes refresh tokens server-side), automatic logout after 30 minutes of inactivity, every sign-in/out logged.
Secrets	Browser has only the public anon key. Service-role key, OAuth client secrets, AI key live as Edge Function secrets. OAuth tokens are encrypted in Supabase Vault and readable only by service_role functions.
OAuth integrity	State parameter is HMAC-signed, bound to user and workspace, expires in 10 minutes (CSRF protection).
Edge Functions	Each re-verifies the JWT and the specific permission server-side; input is validated; errors are generic; CORS allows only your listed origins; Postgres-backed rate limits.
Privilege hardening	The anon role has no table or function privileges; functions are executable only by authenticated users or the service role; secret-handling functions are service-role only (migration 0007).
Admin allow-list	Only approved Gmail accounts can sign in to Workspace; any other Google account is rejected by the database at account creation and the login page explains why. Only the owner can approve or remove Admins.
Data retention	Attendance, activity and automation history are deleted after 7 days; login history after 7 to 30 days (your choice); read notifications after 30 days.
Least visibility	Team Members can read only their own tasks and activity; Admins can read everything in the workspace.
Project separation	Office, Instagram and YouTube live in separate Supabase projects; a breach of one does not expose the others. Their service keys are held only in Vault of project A and used only by project-gateway with a table/function whitelist. Projects B, C, D close sign-ups after the first admin (D has no client access at all).
Frontend delivery	Protected pages are code-split and downloaded only after login; production HTML ships a strict Content-Security-Policy; _headers adds frame, MIME and referrer protections on Netlify/Cloudflare. The files themselves are public by nature, so security never depends on hiding them.
Desktop shell	contextIsolation and sandbox on, no Node in the page, bundled app served from a custom scheme with path-traversal protection, navigation locked to the app, one validated IPC call, system browser for Google sign-in, PKCE.
Storage	Private bucket with membership-scoped policies and signed URLs; upload size limits; image type check for avatars.
Cache and logout	No data is rendered without a valid session; data comes only from authenticated, RLS-checked queries; service worker never caches API responses.
Audit	activity_logs, security_events, integration_logs, automation_logs, call history.

5.2 Why a private URL is not security

A hidden URL or private GitHub repository does not protect data. The site itself is public static files; protection comes from authentication, RLS and Storage policies. Even if someone copies the frontend, they cannot read

data without a valid session and permission.

5.3 Threats and responses

Threat	Response
Someone signs up with a random email	Blocked by handle_new_user
Member calls the API directly to read others' private data	RLS policy denies; restricted table
Member edits their own name/role/phone via API	Guard triggers raise an error
Stolen password/OTP session when 2FA is on	mfa_gate denies data without aal2
Forged OAuth callback	Signed state check fails
Browser exposes secrets	No secrets in frontend; Vault + function secrets
Forgotten logged-in computer	Idle logout, sign out everywhere
Brute-force login	Supabase Auth rate limits (configure in Dashboard)

6. Module notes

6.1 Integrations

Providers: Google, YouTube, GitHub, Google Drive, Google Calendar, Gmail (OAuth), Supabase (built-in), Webhooks and Future APIs (configuration only, no secrets). Each card shows status, connected account, last sync, granted scopes, last error and logs. Adding another provider means adding one entry to the function's provider table and the frontend list.

6.2 YouTube Automation

- **Separate backend:** all YouTube data and tokens live in the YouTube Automation Supabase project (D); Workspace shows and controls it only through project-gateway.
- **Data:** subscribers, channel views, video count snapshots; per-video views, watch time, average view duration and percentage (retention), likes, comments, shares, subscribers gained, and thumbnail CTR when the Analytics API provides it for your channel.
- **Views:** Overview (KPIs, recent views chart, top videos, Shorts vs long-form), Videos (filter, sort, compare up to three), AI Studio (ideas, hooks, titles, scripts, thumbnail concepts, SEO, upload timing, Shorts ideas, content plan, growth, content and competitor research).
- **Not included:** actual thumbnail image generation (needs an image API key and provider choice). Concepts are produced as text.
- **Shorts detection** uses a 60-second heuristic that you can change in youtube-sync.

6.3 Instagram, Agents, Office

- Instagram: backend link, execution history and a documented plan to analyze, reuse and integrate your existing Lexon Automation project without rebuilding it.
- Agents: tables, permission, navigation and an honest empty state; no fake agents.
- Office: a separate virtual environment. Workspace never copies real team members into it. Workspace only links its backend, shows status/events and manages Office AI/API integrations.

6.4 Assistant (Google Gemini)

The Home assistant card opens a chat panel. The name is changeable per user. Messages go to the **assistant** Edge Function, which calls Gemini with the Gemini key stored in Vault. Context (tasks, projects, team, activity)

is read with the caller own session, so a Team Member can never get answers about data they may not see. It behaves like a normal assistant (greetings, general questions, suggestions, any language). Voice input and read-aloud use the browser speech features (best in Chrome and Edge; not available in the desktop shell for input).

6.5 Unified notification center

The bell shows personal notifications (messages, calls, task assignments, announcements) plus workspace activity (files, automations, team, agents, assistant). Events from the Instagram and Office projects are pulled in through the gateway. Admins can send announcements to everyone, Team Members, Admins or selected people.

6.6 Invitation lifecycle

Invite creates an allow-list entry, creates the account, emails an invitation and returns a one-time code and link for the admin to share (useful because default Supabase email is rate limited). Status comes from the real account: Not registered, Pending (account exists, never signed in), Joined. Resend creates a new code; Revoke removes a pending invite. Flow: Invite, Receive, Login (email code in Lexon Team), Join Team.

7. Four Supabase projects and the gateway

The final backend separation is exactly: Workspace + Team = ONE shared Supabase; Office = separate; Instagram Automation = separate; YouTube Automation = separate. Workspace and Team are two frontends of project A, so they share auth, database, RLS, realtime, messages, tasks, notifications, files, team data, activity and calls with no sync code.

7.1 How Workspace talks to B, C and D

Separate Supabase projects have separate users, so their tokens are not interchangeable. Instead of giving the browser three more keys, Workspace uses one Edge Function in project A, **project-gateway**:

- The browser sends its normal Workspace session. The gateway checks the JWT and the permission in project A.
- The linked project URL and service key were saved once (Integrations, Link project) into Vault by the same function; they never exist in the browser or in plain tables.
- Only whitelisted tables and functions per project are reachable; writes are limited (for example only ig_rules for Instagram); requests are rate limited.
- ping verifies the connection and schema; pull_events copies new Instagram/Office events into Workspace history. Office is never fed with real Workspace team data.

7.2 SQL for each project

File	Project	Notes
LEXON_A_workspace_team.sql	A (shared)	Migrations 0001 to 0007 in one file. Run once.
LEXON_B_office.sql	B (Office)	Closed sign-up (first user = admin). Virtual office members, agents, layout and events.
LEXON_C_instagram.sql	C (Instagram)	Baseline schema (rules, events, actions, threads, analytics, logs, Vault token functions). To be reconciled with your existing project.
LEXON_D_youtube.sql	D (YouTube)	No client policies at all; only the service role. Plus Edge Functions in lexon-backends/youtube.

8. Web, PWA and desktop

One codebase: apps/web (Workspace) and apps/team (Team) are static builds. The deployed website is for computers that are not yours; the desktop app is for your own Windows/macOS machine. Both use the same online backend, so messages, files, tasks, team updates and automation data stay consistent.

- **Separate URLs:** Workspace and Team are deployed separately (own GitHub Pages / hosting URL) and use the same VITE_SUPABASE_URL and anon key.
- **Desktop shell (apps/desktop):** Electron wraps the same build. Build with npm run dist:win on Windows or npm run dist:mac on a Mac. Google sign-in opens the system browser and returns through the lexon:// deep link (PKCE); add lexon://app/** to Supabase redirect URLs.
- **PWA:** manifest + minimal service worker (static assets only). Installable on phones and desktops from the browser.
- **Not included:** auto-update and code signing (needs Apple/Windows certificates).

9. Environment values you must add yourself

Where	Value	Public?
apps/web/.env and apps/team/.env	VITE_SUPABASE_URL, VITE_SUPABASE_ANON_KEY (project A). Optional VITE_TURN_URL/USER/PASS	Public values
Project A function secrets	ALLOWED_ORIGINS, APP_URL, TEAM_APP_URL, OAUTH_STATE_SECRET, GOOGLE_CLIENT_ID/SECRET (and GITHUB_* if used)	SECRET
Project D function secrets	GOOGLE_CLIENT_ID/SECRET, OAUTH_STATE_SECRET, WORKSPACE_URL, ANTHROPIC_API_KEY, optional AI_MODEL	SECRET
Workspace UI (Integrations)	Project URL + service role key of B, C and D, entered once into Link project (stored in Vault)	SECRET
Workspace UI (Integrations)	Gemini API key (assistant) and Office AI/API keys, entered in the UI and stored encrypted in Vault	SECRET
Supabase Auth, SMTP	Custom SMTP provider (Resend, Brevo, etc.) so invitation and login emails are reliable	SECRET
Supabase dashboards	Google provider, redirect URLs (web, Team, lexon://app/**), email template with the code token, TOTP MFA, rate limits; disable sign-ups on B, C, D after first admin	n/a
Google Cloud	OAuth client; enable YouTube Data API v3 and YouTube Analytics API; redirect URIs for project A integrations-oauth and project D youtube-oauth	n/a

10. Setup and deployment (summary)

Full command-by-command steps are in **docs/MANUAL_SETUP.md**. Order:

- 1. Create a Supabase project; copy URL and anon key into each app's .env.
- 2. Run LEXON_A_workspace_team.sql in project A. For Office, Instagram and YouTube create separate projects and run their own SQL (B, C, D).
- 3. Configure Auth: Google provider, URL configuration (Workspace + Team URLs), email template containing the code token, enable TOTP MFA, review rate limits.
- 4. Log in to Workspace first with your Google account: you become Owner.
- 4b. Settings, Admin access: approve additional Admin Gmail accounts (owner only). Settings, Integrations: add the Gemini key for the assistant.

- 5. Deploy Edge Functions and set secrets (APP_URL, TEAM_APP_URL, OAUTH_STATE_SECRET, Google/GitHub client credentials, ANTHROPIC_API_KEY).
- 6. In Google Cloud enable YouTube Data API v3 and YouTube Analytics API; add project D youtube-oauth URL as a redirect URI. Deploy project D functions and set its secrets.
- 7. Deploy both apps (GitHub Pages workflow included) and add their URLs to Supabase Auth.
- 8. Turn on 2FA for your own account (Settings, Security).
- 9. Invite members from Team, Add member.

Note. Never commit .env files or the service-role key. Only VITE_ variables (public) belong to the frontend build.

11. Using the system day to day

Task	How
Add a member	Workspace, Team, Add member (name, email, phone, role). They receive an invitation and log in to Lexon Team with an email code.
Change what roles can do	Settings, Permissions (Owner only)
Assign work	Tasks, Create task, choose project and assignees; members see it in Team Home and get a notification
Share files	Files, upload into folders; chat attachments via Messages
Call someone	Open a direct conversation, tap the phone or video icon
Connect YouTube	Integrations, YouTube, Connect; Automations, YouTube, Sync now
Sign out a lost device	Settings, Security, Sign out everywhere

12. Troubleshooting

Symptom	Likely cause and fix
Blank page / env message	Missing VITE_SUPABASE_URL or VITE_SUPABASE_ANON_KEY
Team login says not authorized	Email not invited, or the email template lacks the code token
Cannot read data after enabling 2FA	Session is not aal2; sign in again and enter the code
Integration connect fails	Redirect URI mismatch or missing secrets; check function logs in Supabase
YouTube CTR shows a dash	Analytics API did not return thumbnail CTR for this channel; see the sync note in Integrations logs
No refresh token from Google	Remove the app in your Google account permissions and connect again
Calls connect but no audio/video	Strict NAT: configure a TURN server (VITE_TURN_*)
SQL error on first run	Send the error text; likely an extension (Vault) or ordering issue
Member cannot see something	Check Settings, Permissions for their role

13. Limitations and known gaps (read this)

- **Untested build:** code, SQL and Edge Functions were not executed in the sandbox. Run npm install and npm run build, run the SQL, deploy functions, then fix first-run errors. Use the test checklist in MANUAL_SETUP.md.

- **Call signaling channel** is a public Realtime broadcast channel keyed by an unguessable call id and carries only SDP/ICE. For stricter setups, move it to private channels with Realtime authorization policies.
- **TURN credentials** in VITE_ variables are visible to users; use a provider with time-limited credentials for production.
- **Avatars bucket is public-read** (profile photos). Files are private.
- **Login history** is written by the client (no IP address); it is a user-facing log, not forensic evidence. Use Supabase Auth logs for forensics.
- **Passkeys/WebAuthn** are not available in Supabase Auth; TOTP is used.
- **Idle logout** is client-side; combine with Supabase session settings (some are plan-dependent).
- **Assistant:** depends on your Gemini key, quota and the model name; voice input needs Chrome or Edge and is not available inside the desktop shell. Default Supabase email is rate limited: configure SMTP, or share the one-time code shown after inviting.
- **Retention:** pg_cron must be enabled for the nightly cleanup; otherwise the cleanup runs when an Admin opens the app.
- **YouTube metrics availability** (for example thumbnail CTR) depends on what the Analytics API returns for your channel; the app shows a dash instead of inventing numbers.
- **Not built:** thumbnail image generation, Instagram integration (baseline schema only), Office app, Agents logic, group calls, screen share, automated tests, desktop auto-update and code signing.
- **Service keys of B, C, D** are pasted once into Workspace and held in Vault of project A. Anyone with the integrations.manage permission can replace them; keep that permission to the Owner.
- **Frontend files are public by nature.** Hiding or renaming routes gives no protection; protection comes from authentication, RLS, Storage policies, 2FA and server-side checks.

14. Roadmap

Next	Details
Stabilize	First build, run SQL, fix errors, add automated tests for RLS, permissions, task assignment, messaging, calls
Instagram	Share the existing Lexon Automation project; analyze, reuse, move logic to an Edge Function/worker, add rule editor and logs
Office	Mount your Office project in its own environment
Agents	Manager/Agent system on the prepared tables
Calls	Group calls, screen sharing, private signaling channels, TURN
Desktop	Code signing, auto-update, Team desktop release
AI	Office AI usage of stored keys; thumbnail image generation with a chosen provider